

为什么需要 I/O 接口电路?

- ① 微机的外部设备多种多样
- ② 各外设的工作原理、驱动方式、信息格式、工作速度方面彼此差别很大
- ③ 各外设不能与 CPU 直接相连, 必须通过中间电路

什么是 I/O 接口(电路)?



- ① I/O 接口是位于系统与外设间, 用来协助完成数据传送和控制任务的逻辑电路
- ② PC 机系统板的可编程接口芯片、I/O 总线槽的电路板(适配器)都是接口电路

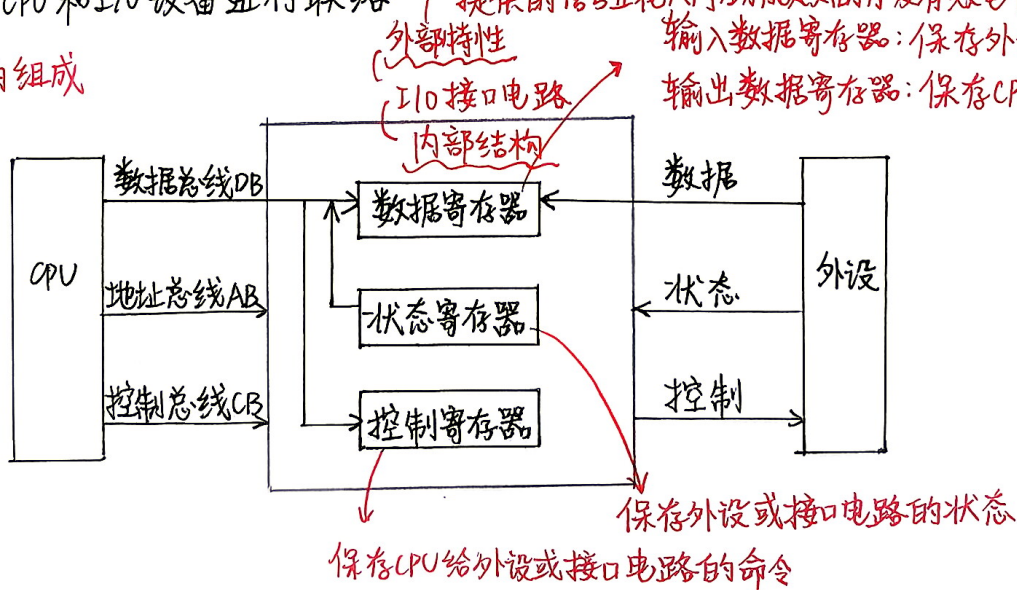
什么是微机接口技术?

- ① 处理微机系统与外设间联系的技术
- ② 有软、硬结合的特点
- ③ 根据应用系统的需要, 使用并构造相应的接口电路, 编制配套的接口程序, 支持和连接有关设备

I/O 接口的主要功能?

- ① 对输入输出数据进行缓冲和锁存
输出接口有锁存环节, 输入接口有缓冲环节
实际的电路常见: 输出锁存缓冲环节, 输入锁存缓冲环节
- ② 对信号的形式和数据的格式进行变换. 微机直接处理: 数字量、开关量、脉冲量
主要体现在引脚上, 分成两侧信号, 面向 CPU 一侧的信号用于 CPU 连接, 主要是数据、地址、控制信号; 面向外设一侧的信号用于外设连接, 提供的信号五花八门, 功能定义、时序及有效电平差异较大
- ③ 对 I/O 端口进行寻址
- ④ 与 CPU 和 I/O 设备进行联络

简单 I/O 接口的组成



接口电路芯片的分类

接口电路的核心部分往往是一块或数块大规模集成电路芯片(接口芯片):

① 通用接口芯片

支持通用的数据输入输出和控制的接口芯片

② 面向外设的专用接口芯片

针对某种外设设计 与该种外设接口

③ 面向微机系统的专用接口芯片

与CPU和系统配套使用以增强其总体功能

许多接口电路具有多种功能和工作方式,可通过编程方法选定其中一种

接口需进行物理连接,也需编写接口软件(2类):

① 初始化程序段——设定芯片工作方式

② 数据交换程序段——管理、控制、驱动外设,负责外设与系统间信息交换

接口电路占用的I/O端口有2类编排形式:

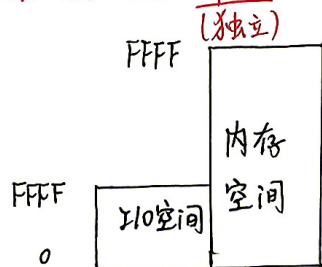
① I/O端口单独编址

I/O地址空间独立于存储地址空间,如8086/8088

② I/O端口与存储器统一编址

它们共享一个地址空间,如M6800

(1) I/O 端口单独编址 (80x86 采用)



优点: I/O 端口的地址空间独立

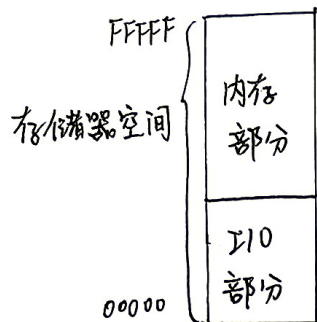
控制和地址译码电路相对简单

专门的 I/O 指令使程序清晰易读

缺点:

I/O 指令没有存储器指令丰富。

(2) I/O 端口与存储器统一编址



优点: 无需专门的 I/O 指令

I/O 数据存取与存储器数据一样灵活

缺点:

I/O 端口要占用部分存储器地址空间

程序不易阅读 (不易分清访存和访问外设)

8086 / 8088 的输入输出指令

输入指令: 将外设数据传送给 CPU 内的 AL/AX

IN	AL, 8 位立即数 (例如 60H); 字节输入, 将对应端口数据读入 AL (直接寻址, 0~255)
IN	AL, DX; 字节输入, 将 DX 中 16 位地址对应的端口的数据读入 AL (间接寻址, 端口号 > 255)
IN	AX, 8 位立即数; 字输入, 将小端口及其后的共 2 个字节读入 AX (高字节是大端口号, 低字节是小端口号)
IN	AX, DX; 字输入, 将 DX 对应的端口的 16 位数据读入 AX

用 AX 16 位传输, 会占用 2 个相邻的端口号, 低字节对应小端口号, 高字节对应大端口号

输出指令: 将 CPU 内的 AL/AX 数据传送给外设

OUT	8 位立即数, AL; 字节输出, 将 AL 中的数据写入对应端口 (直接寻址, 0~255)
OUT	DX, AL; 字节输出, 将 AL 中的数据写入 DX 对应端口 (间接寻址, > 255)
OUT	8 位立即数, AX; 字输出, 将 AX 中的 16 位数据写入 18 对应的连续 2 个端口
OUT	DX, AX; 字输出, 将 AX 中的 16 位数据写入 DX 对应的 2 个连续端口

8088 只能通过输入输出指令与外设进行数据交换, 呈现给程序员的外设是端口 (PORT),

即 I/O 地址

8086 用于寻址外设端口的地址线为 16 条, 端口最多有 $2^{16} = 65536$ (64K) 个, 端口号为 0000H ~ FFFFH

★ 每个端口用于传送一个字节的外设数据

8088 / 8086 的端口有 64K 个, 无需分段, 有 2 种寻址方式:

直接寻址: 只用于寻址 00H ~ FFH 前 256 个端口, 操作数表示端口号

间接寻址: 可用于寻址全部 64K 个端口, DX 寄存器的值就是端口号

大于 FFH 的端口只能采用间接寻址方式

小结.

☆ 若输入输出一个字节, 则利用 AL 寄存器; 若输入输出一个字, 则利用 AX 寄存器

☆ 输入一个字, 实际上是从连续 2 个端口输入 2 个字节, 分别送 AL (对应低地址端口) 和 AH (对应高地址端口)

输出一个字, 实际上是将 AL (对应低地址端口) 和 AH (对应高地址端口) 2 个字节的內容输出给连续 2 个端口

例. IN 指令实例 (从 20H 端口输入一个字)

① 方法 1: 直接寻址, 字量输入

```
IN AX, 20H
```

② 方法 2: 间接寻址, 字量输入

```
MOV DX, 20H
```

```
IN AX, DX
```

③ 方法 3: 直接寻址, 字节量输入

```
IN AL, 20H
```

```
MOV AH, AL
```

```
IN AL, 20H
```

④ 方法 4: 间接寻址, 字节量输入

```
MOV DX, 20H
```

```
IN AL, DX
```

```
MOV AH, AL
```

```
DEC DX
```

```
IN AL, DX
```

例. OUT 指令实例 (向 300H 端口输出一个字节)

唯一的方法: 间接寻址, 字节量输出

```
MOV AL, BVAR ; BVAR 是字节变量
```

```
MOV DX, 300H
```

```
OUT DX, AL
```

I/O 地址的译码方法与存储器地址的译码方法一样, 但有它的特点:

- 部分译码时, 通常中间地址线不连接
- 部分译码也有最低地址线不连接的情况
- 每个接口电路通常只占用几个 I/O 地址, 这时可利用基本逻辑门电路进行地址译码
- 除采用译码器、门电路译码外, I/O 地址译码亦常采用可编程逻辑器件 PLD
- 为给系统一定的选择余地, 有些接口电路利用比较器、开关或跨接器等进行多组 I/O 地址译码

CPU与外设之间数据传输的控制方式

- 程序控制下的数据传送——通过CPU执行程序中的I/O指令完成传送,又分为
 - 无条件传送
 - 查询传送
- 中断传送
- 直接存储器存取(DMA)——传送请求由外设向DMA控制器(DMAC)提出,后者向CPU申请总线,最后DMAC利用系统总线来完成外设和存储器间的数据传送

①

(1) 无条件传送方式

在CPU与慢速变化的设备交换数据时,可认为它们总处于“就绪”状态,随时可进行数据传送,这就是无条件传送,或称立即传送、同步传送

接口与操作都十分简单,适用于LED数码管、按键、按钮等简单设备

这种传送有前提: 外设必须随时就绪

例. MOV DX, 160H
 IN AL, DX

例. MOV DX, 160H ; 把端口地址160H存入DX寄存器
 MOV AL, [BX] ; 把内存中BX寄存器指向地址里的1字节数据读入AL
 OUT DX, AL ; 把AL中数据写入DX指向的端口

例.

实现拨动开关,
LED状态切换,
并且拨几个开关
都无所谓

NEXT:

MOV DX, 8000H ; DX指向数据端口
IN AL, DX ; 从输入端口读开关状态
NOT AL ; 反相
OUT DX, AL ; 送输出端口显示
CALL DELAY ; 调用子程序延时
JMP NEXT ; 重复

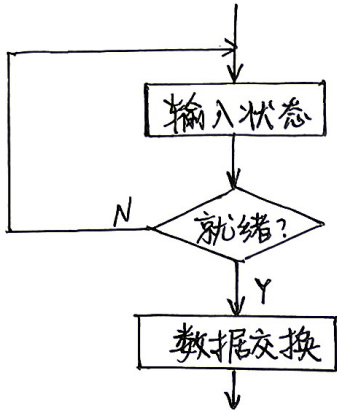
(2) 条件传送方式

就绪

CPU 要先了解(查询)外设的工作状态,然后在外设可以交换信息的情况下实现数据输入或输出

对多个外设的情况, CPU 按一定顺序依次查询(轮询), 先查询的外设将优先进行数据交换

特点: 工作可靠, 适用面宽, 传送效率低



查询环节 { 寻址状态口
读取状态寄存器的标志位
若非就绪就继续查询直至就绪

传送环节 { 寻址数据口
从端口读
是输入则通过输入指令从数据端口读入数据
是输出则通过输出指令向数据端口输出数据
往端口写

例. 查询输入接口 (8000H, 8001H 规定好了).

三态缓冲器

MOV DX, 8000H ; DX 指向状态端口

STATUS:

IN AL, DX ; 读状态端口中的信息到 AL

“与”

但不返回值, 也不改操作数

TEST AL, 01H ; 测试, 仅保留最后1位 D₀

JZ STATUS ; D₀=0, 未就绪, 继续查询

INC DX ; D₀=1, 就绪, DX 指向数据端口

IN AL, DX ; 从端口读值到 AL

查询输出接口

锁存器

MOV DX, 8000H ; DX 指向状态端口

STATUS:

IN AL, DX ; 读状态端口中的信息到 AL

TEST AL, 80H ; 测试, 仅保留最高位 D₇

JNZ STATUS ; D₇≠0 (D₇=1), 未就绪, 继续查询

INC DX ; D₇=0, 就绪, DX 指向数据端口

MOV AL, BUF ; BUF = 变量

OUT DX, AL ; 将数据输出给数据端口

PPT 上有图, 可看看.

例. 查询方式的 EPROM 编程

NEXT:

立即数切不可直接写进 [BX][SI][DI][BP] 等内存间接寻址单元

MOV AL, 55H ; 写入内容 55H

MOV [BX], AL ; 将内容写入 BX 所指的存储单元

NOP

NOP

} 空操作指令, 延时 (占1字节内存空间, 消耗1个CPU时钟周期)

NEXT1:

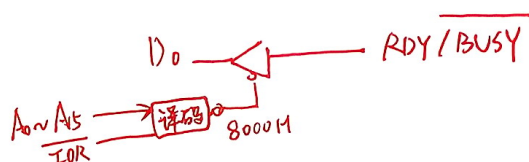
IN AL, DX ; 查询状态口 (读状态口信息到 AL)

TEST AL, 01H ; 测试 D₀

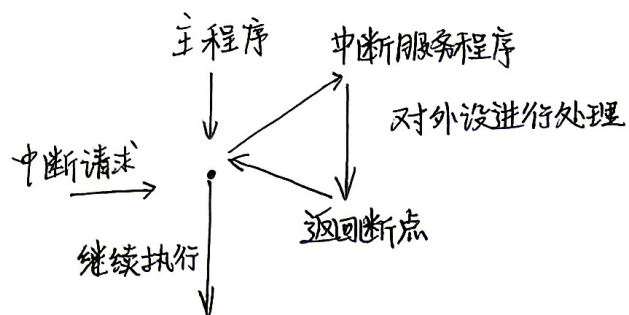
JZ NEXT1 ; D₀=0, 芯片还在写入, 再回去一直查询, 直至

INC BX ; D₀=1, 写完, 指针移动

LOOP NEXT ; 循环至写完全部字节



② 中断传送方式



CPU在执行程序中,被内部或外部事件打断,转去执行一段预先安排好的中断服务程序;服务结束后返回原来断点,继续执行原来程序

中断传送是效率更高的程序传送方式

CPU对请求的检测是有规律的,一般在每条指令最后一个时钟周期采样中断请求输入引脚
中断请求是外设随机向CPU提出的,但

进行传送的中断服务是预先设计好的

③ DMA 传送方式

- 希望克服程序控制传送的不足：外设 $\leftrightarrow$ CPU $\leftrightarrow$ 存储器
- 直接存储器存取： 外设 $\leftrightarrow$ 存储器
- ★ CPU 释放总线，由 DMA 控制器管理

DMA 传送的工作过程？

CPU 对 DMA 控制器进行初始化设置



外设、DMAC、CPU 三者通过应答信号建立联系：CPU 将总线交由 DMAC 控制



DMA 传送：

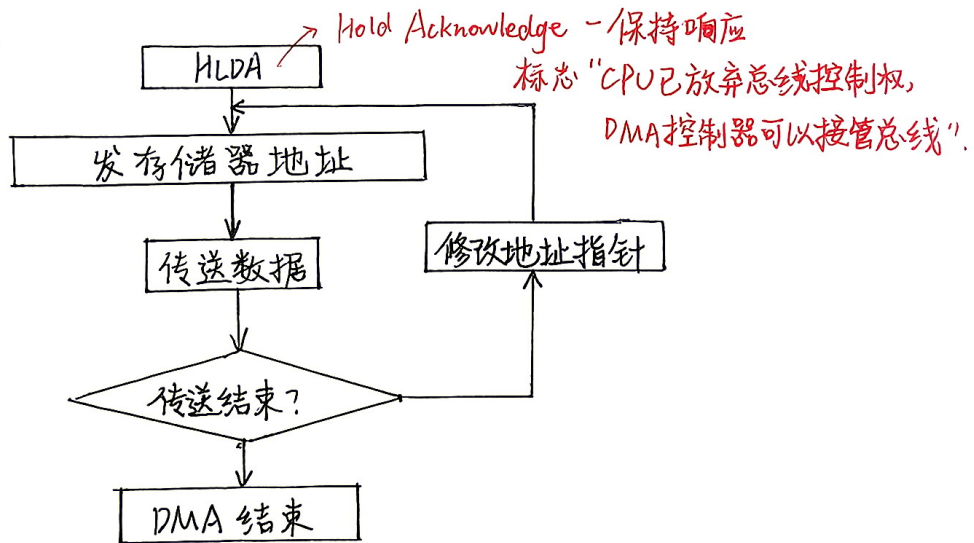
DMA 读存储器：存储器 $\rightarrow$ 外设

DMA 写存储器：存储器 $\leftarrow$ 外设



自动增减地址和计数，判断传送完成与否

DMA 传送流程？



传送方式的比较 · 小结

无条件传送: 慢速外设需与CPU保持同步

查询传送: 简单实用, 效率低

中断传送: 外设主动, 可与CPU并行工作, 但每次传送需大量额外时间开销

DMA传送: DMAC 控制, 外设直接与存储器进行数据传送, 适合大量、快速数据传送

并行接口技术

并行接口概述

并行数据传输方式

- 以计算机的字长, 通常是8位、16位、32位为传输单位, 一次传递一个字长的数据
- 适合于外部设备与微机之间进行近距离、大量、快速的信息交换
例如: 微机和并行接口打印机、磁盘驱动器
- 微机系统中最基本的信息交换方法
例如: 系统板上各部件之间, 接口电路板上各部件之间

可编程并行接口芯片8255A

- 具有多种功能的可编程并行接口电路芯片
 - △ 最基本的接口电路: 三态缓冲器和锁存器
 - △ 与CPU间、与外设间的接口电路: 状态寄存器和控制寄存器
 - △ 端口的译码和控制电路、中断控制电路
- 共24个外设引脚, 分3个端口、2组控制
- 共3种输入输出工作方式

(1) 外设数据端口

端口A: PA0 ~ PA7

A组, 支持工作方式0、1、2

常作数据端口, 功能最强大

端口B: PB0 ~ PB7

B组, 支持工作方式0、1

常作数据端口

端口C: PC0 ~ PC7

C组, 支持工作方式0

{ A组控制高4位 PC4 ~ PC7
B组控制低4位 PC0 ~ PC3

可作数据、状态、控制端口。分为2个4位, 每位可独立操作
控制最灵活, 最难掌握

(2) 与处理器接口

D0 ~ D7 数据线

A0 ~ A1 地址线

$\overline{CS}$ 片选信号

$\overline{RD}$ 读信号

$\overline{WR}$ 写信号

RESET 复位信号

$\overline{CS}$	A1	A0	I/O地址	读操作 $\overline{RD}$	写操作 $\overline{WR}$
0	0	0	60H	读端口A	写端口A
0	0	1	61H	读端口B	写端口B
0	1	0	62H	读端口C	写端口C
0	1	1	63H	非法	写控制字

8255A 是可编程接口芯片

8255A 的控制字

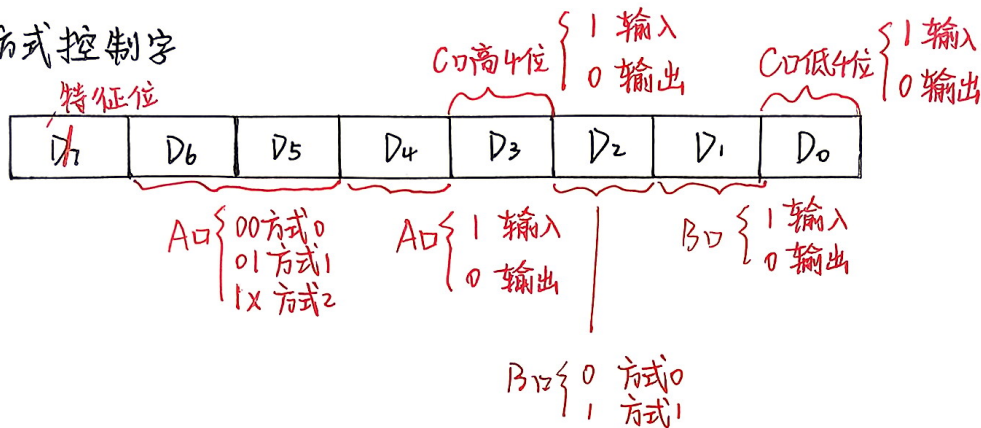
8255A 有 3 种工作方式可供选择, 使用前应先对芯片进行初始化, 决定芯片的端口是处于输入数据状态还是输出数据状态以及每个端口的工作方式

工作方式和工作状态的建立由向 8255A 的控制口写入相应的控制字完成

8255A 共有 2 个控制字, 即工作方式控制字、对 C 口置位/复位控制字

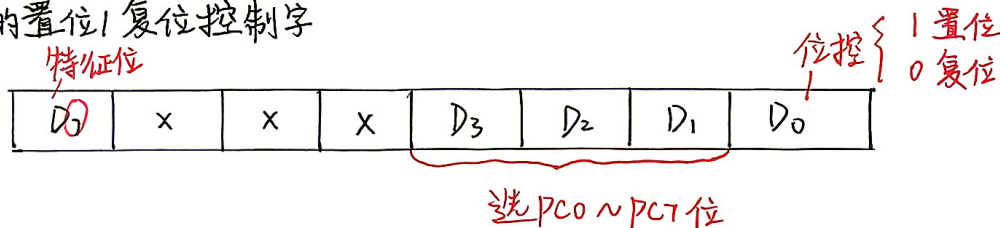
这 2 个控制字使用同一个端口地址, 由控制字的最高位 D_7 选择, 若 $D_7=1$, 则控制字为工作方式控制字; 若 $D_7=0$, 则控制字为 C 口置位/复位控制字

① 工作方式控制字



A 口有 3 种工作方式: 0, 1, 2 B 口有 2 种工作方式: 0, 1 C 口有 1 种工作方式: 0

② C 口的置位/复位控制字



8255A 的工作方式

① 方式 0: 基本输入/输出方式

适用于无条件传送或查询传送, 如开关状态输入、LED 显示输出等。

输出信息时, 端口有锁存功能; 输入信息时, 端口只有缓冲功能而无锁存功能

若 A 口、B 口都工作在方式 0, 则存在独立的 2 个 8 位并行口 (端口 A、B) 和 2 个 4 位并行口 (C 口的高 4 位、低 4 位)

四个并行口中任一个都可以选择输入或输出, 由此得到 16 种组合

② 方式 1: 选通输入/输出方式

适用于查询和中断方式的接口电路

数据的输入/输出操作要在选通信号控制下完成。

利用专用的中断请求和联络信号线控制数据传送, 数据的输入/输出有锁存能力, 只有端口 A 或 B 用于数据传送, 端口 C 的大部分 I/O 线作联络控制线使用

③ 方式 2:

适用于双向传送数据的外设, 适用于查询和中断方式的接口电路

仅 A 口可以工作在这种方式

方式1 输入联络信号

一对应答
联络信号
为了可靠
地输入
数据

$\overline{STB}$: 选通信号, 低电平有效 ($PC4^A$) ($PC2^B$)

由外设提供的输入信号, 当其有效时, 将输入设备送来的数据锁存至8255A的输入锁存器

IBF : 输入缓冲器满信号, 高电平有效 ($PC5^A$) ($PC1^B$)

8255A输出的联络信号, 当其有效时表示数据已锁存在输入锁存器

$INTR$: 中断请求信号, 高电平有效 ($PC3^A$) ($PC0^B$)

8255A输出的联络信号, 可用于向CPU提出中断请求, 要求CPU读取外设数据

8255A的中断由中断允许触发器 $INTE$ 控制 (置位允许中断, 复位禁止中断)

对 $INTE$ 的操作通过写入端口C的对应位实现,

$INTE$ 触发器对应端口C的位是作应答联络信号的那一位, 只要对该位置位/复位

即可控制 $INTE$ 触发器 (选通输入方式下: 端口A的 $INTEA$ 对应 $PC4$, 端口B的 $INTEB$ 对应 $PC2$)

方式1 输出联络信号

一对应答
联络信号
为了可靠地
输出数据

$\overline{OBF}$: 输出缓冲器满信号, 低电平有效 ($PC7^A$) ($PC1^B$)

8255A输出给外设的一个控制信号, 当其有效时表示CPU已把数据输出给指定的端口, 外设可以取走

$\overline{ACK}$: 响应信号, 低电平有效 ($PC6^A$) ($PC2^B$)

外设的响应信号, 指示8255A的端口数据已由外设接收

$INTR$: 中断请求信号, 高电平有效 ($PC3^A$) ($PC0^B$)

当输入设备已接收数据后, 8255A输出该信号向CPU提出中断请求, 要求CPU继续提供数据

选通输出方式下: 端口A的 $INTEA$ 对应 $PC6$, 端口B的 $INTEB$ 对应 $PC2$

方式2 双向方式

方式2将方式1的选通输入输出功能组合成一个双向数据端口,可发送、接收数据.

仅端口A可以工作于方式2,需利用端口C的5个信号线

PC3 - INTR
PC7 - $\overline{OBF}$
PC6 - $\overline{ACK}$
PC4 - $\overline{STB}$
PC5 IBF

方式2的输入过程与方式1的输入方式一样

方式2的输出过程与方式1的输出方式有一处不同:

不在 $\overline{OBF}$ 有效时向外设输出数据,而是在外设提供响应信号 $\overline{ACK}$ 时才输出数据

初始化编程：一个方式控制字

采用控制I/O地址：A1A0=11

工作过程中，通过数据端口对外设数据进行读写

数据读写利用端口A、B、C的I/O地址，依次为00、01、10

IBM PC/XT机，端口A、B、C和控制端口的I/O地址依次为60H、61H、62H、63H

例. 写入方式字

要求：A口：方式1输入，C口高4位：输出，C口低4位：输入，B口：方式0输出

初始化程序段

```
MOV DX, 0FFEH ; 假设控制端口为FFEH
MOV AL, 0B1H ; 方式控制字 B1H = 10110001B
OUT DX, AL ; 送到控制端口
```

例. 读写数据端口

初始化编程后，将数据端口作为输入接口时，执行输入IN指令，从输入设备获得外设数据

将数据端口作为输出接口时，执行输出OUT指令，把CPU的数据送给输出设备

8255具有锁存输出数据的能力：

对输出方式的端口同样可以输入。

读取的不是外设的数据而是上次CPU给外设的数据

利用输出锁存能力，可以实现按位输出控制

把输出端口B的PB7位进行“置位”操作

```
MOV DX, 0FFAH ; 假设B端口为FFAH
IN AL, DX ; 读出B端口原输出内容
OR AL, 80H ; 使PB7=1
OUT DX, AL ; 输出改后的内容
```

例. 读写端口C

C端口被分为2个4位端口, 2个端口只能以方式0工作, 可分别选择为输入/输出
在控制上, C端口上半部与A端口编为A组, C端口下半部与B端口编为B组

当A、B端口工作在方式1或方式2时, C端口的部分或全部引脚将被征用。
其余引脚仍可设定工作在方式0

端口C的数据输出有2种方法:

方法一: 通过端口C的I/O地址, 向C口直接写入字节数据,

写入C口的输出锁存器并从输出引脚输出但对设为输入的引脚无效

方法二: 通过控制端口, 向C口写入位控字,

使C口的某个引脚输出1或0, 或置位复位内部的中断允许触发器

方式1输入: $\overset{A}{PC4}$ vs $\overset{B}{PC2}$

方式1输出: $\overset{A}{PC6}$ vs $\overset{B}{PC2}$

读取的C端口数据有2种情况:

未被A、B口征用的引脚:

将从定义为输入的端口读引脚输入信息,

将从定义为输出的端口读输出锁存器中的信息.

被A、B口征用为联络线的引脚:

读反映8255A状态的状态字

例. 8255A在IBM PC/XT上的应用

工作在基本输入/输出方式0

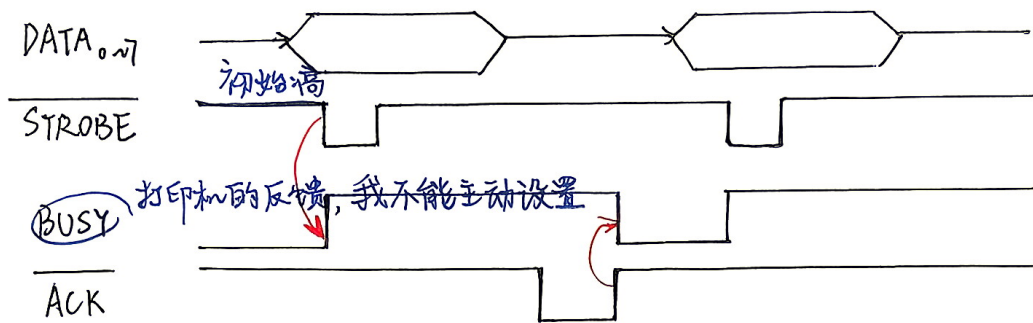
- A口为方式0输入, 读取键盘扫描码
- B口为方式0输出, 控制扬声器
- C口为方式0输入, 读取系统状态和配置

初始化编程:

`MOV AL, 10011001 B; 方式控制字`

`OUT 63H, AL; 把方式控制字写入控制端口`

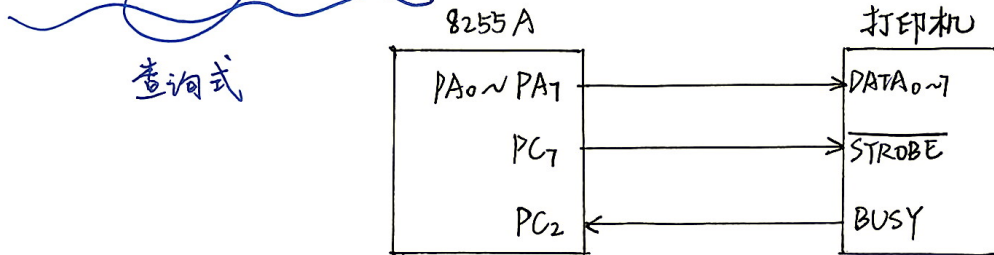
例. 打印机接口的信号与时序



主机把数据送给引脚 $DATA_0 \sim DATA_7$, 同时送出数据选通信号 $\overline{STROBE}$

打印在 $BUSY$ 信号线上发出忙信号, 打印机处理好输入的数据时 { 撤销 $BUSY$
送出响应信号 $\overline{ACK}$

例. 用8255A方式0与打印机接口



控制端口地址: FFEH

A口方式0输出, C口上输出、下输入

初始化:

```
MOV DX, 0FFFEH ; 控制端口地址写入DX
MOV AL, 10000001B ; 写方式控制字
OUT DX, AL ; 给到控制端口
```

```
MOV AL, 00001111B ; 写位控字, 使PC7=1, 不选通
OUT DX, AL ; 给到控制端口
```

打印子程序:

PRINTC PROC

```
PUSH AX ; 保护
PUSH DX ; 保护
```

查询 {

```
PRN: MOV DX, 0FFFCBH ; C口地址写入DX
      IN AL, DX ; 读C口的信息到AL
      AND AL, 04H ; 读PC2的值
      JNZ PRN ; 不是0说明打印机在忙, 跳回去继续查询
```

输出 {

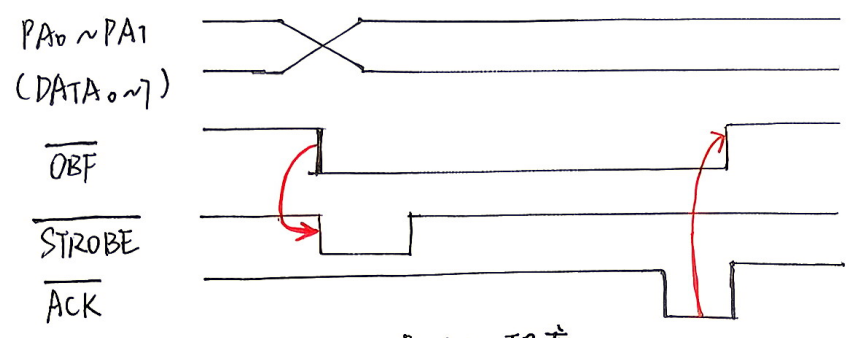
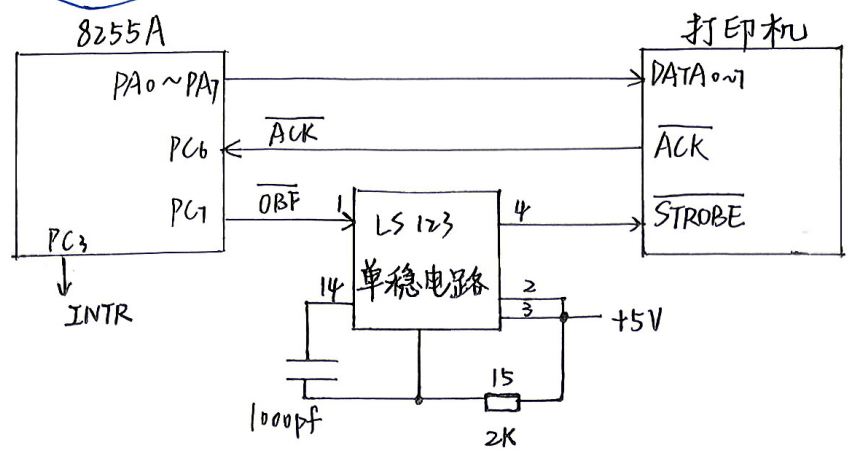
```
      MOV DX, 0FFFBH ; 不忙了, A口地址写入DX
      MOV AL, AH ; OUT要用AL传递8位数据, 输出数据到A口
      OUT DX, AL
```

打印 {

```
      MOV DX, 0FFFEH ; 控制端口地址写入DX
      MOV AL, 00001110B ; 写位控字, PC7=0, 选通
      OUT DX, AL ; 给到控制端口
      NOP ; 让STROBE保持一段时间
      NOP ; 置PC7=1, STROBE不选通
      MOV AL, 00001111B ; 给到控制端口
      OUT DX, AL
      POP DX
      POP AX
      RET
PRINTC ENDP
```

例. 用8255A方式1与打印机接口

中断式



初始化:

打印子程序:

```

MOV DX, 0FFFEH ; 控制端口地址写入DX
MOV AL, 0A0H ; A组, 方式1, 输出
OUT DX, AL ; 给到控制端口
MOV AL, 0CH ; 写位控字, 使PC6=0 (即INTEA=0 禁止A口中断)
OUT DX, AL ; 给到控制端口
(MOV AL, 0DH) ? ; 使PC6=1
(OUT DX, AL)

MOV CX, COUNTER ; 打印字节数送入CX
MOV BX, OFFSET BUFFER ; 取字符串首地址
CALL PRINTS ; 调用打印子程序

PRINTS PROC
    PUSH AX
    PUSH DX ; "保护"
PRINT1:
    MOV AL, [BX] ; 从BX指向内存单元取出一个数据到AL (-字节)
    MOV DX, 0FF81H
    OUT DX, AL ; 从端口A输出
    MOV DX, 0FFFC0H ; 将端口C的地址送入DX
PRINT2:
    IN AL, DX ; 从端口C读1个字节的数据到AL (看状态)
    TEST AL, 80H ; 按位与, 只修改标志位, 看PC7=0
    JZ PRINT2 ; PC7=0, 说明输出缓冲器满但打印机还没取走
    INC BX ; 取走了, BX+1, 弄下一个字节(下一个要打印的数据)
    LOOP PRINT1

    POP DX
    POP AX ; "恢复"
    RET
PRINTS ENDP
    
```

打印
结束